Лабораторная работа № 2

Шифры перестановки: маршрутное шифрование, решётка Флейснера и шифр Виженера

Шуваев Сергей Александрович

Содержание

1 Цель работы **4**

2 Задание **5**

3 Теоретическое введение **6**

4 Выполнение лабораторной работы **7**

5 Выполнение лабораторной работы **9**
5.1 Маршрутное шифрование 10
5.2 Решётка Флейснера 13
5.3 Шифр Виженера 16

6 Выводы **18**

Список литературы **19**

Список иллюстраций

5.1 Общая для всех трёх шифров функция 9
5.2 зеркальная операция 10
5.3 Идея: поворот и орбиты 13
5.4 Заполнение “4 поворота x k^2 прорезей” 14
5.5 Сдвиг с параметром знака 16

1 Цель работы

Изучить и реализовать шифры перестановки.

2 Задание

Реализовать программно все рассмотренные в методических указаниях шифры:

- Маршрутное шифрование (столбцовая перестановка с паролем);
- Шифрование с помощью решётки (решётка Флейснера);
- Шифр Виженера (таблица Виженера).

3 Теоретическое введение

Шифры перестановки преобразуют открытый текст в криптограмму путём перестановки его символов, не изменяя сами символы (в отличие от шифров замены, рассмотренных в лабораторной работе №1). Способ, каким переставляются буквы, является ключом шифра. Такие шифры интересны тем, что криптостойкость обеспечивается не заменой алфавита, а геометрией записи и чтения текста.

4 Выполнение лабораторной работы

Для выполнения заданий использован язык программирования Julia [**julialang?**,**juliadoc?**], как и в предыдущей лабораторной работе.

Маршрутное шифрование. Открытый текст без пробелов разбивается на блоки длины n, всего получается m блоков; при необходимости последний блок дополняется произвольными символами до длины n. Блоки построчно вписываются в таблицу размерности $m \times n$. Ключом шифра являются число n и пароль – слово из n неповторяющихся букв, алфавитный порядок которых задаёт очерёдность считывания столбцов таблицы. Криптограмма получается выписыванием букв по столбцам в этом порядке.

Решётка Флейснера. Способ предложен австрийским криптографом Эдуардом Флейснером в 1881 году. Строится квадрат размерности k, построчно заполняемый числами $1, \ldots, k^2$; последовательными поворотами этого квадрата на 90° по часовой стрелке и присоединением результатов формируется большой квадрат размерности $2k$. Одна и та же цифра встречается в большом квадрате четыре раза – по одному разу в каждой из четырёх копий исходного квадрата, повёрнутых на $0°, 90°, 180°, 270°$. Эти четыре клетки образуют так называемую *орбиту* точки под действием поворота на 90°: физическая “решётка” с прорезями в клетках одной цифры каждой орбиты, будучи наложенной на пустой квадрат и последовательно повёрнутой 4 раза, заполняет текстом весь большой квадрат целиком, не оставляя пропусков и не затрагивая одну клетку дважды. После заполнения большого квадрата криптограмма, как и в маршрутном шифровании, считывается по столбцам в порядке, заданном паролем – в данном случае длиной $2k$

букв.

Шифр Виженера. Опубликован французским криптографом Блезом Виженером в 1585 году в «Трактате о шифрах»; считался нераскрываемым до 1863 года, когда его взломал австриец Фридрих Казиски. Открытый текст разбивается на блоки длины n (длина пароля); в каждом блоке i-я буква циклически сдвигается по алфавиту на число позиций, равное номеру i-й буквы пароля в алфавите. Реализация с таблицей Виженера (строки которой – алфавит, циклически сдвинутый на очередную позицию) эквивалентна прямому сложению номеров букв открытого текста и пароля по модулю мощности алфавита [**vigenere_cipher?**].

5 Выполнение лабораторной работы

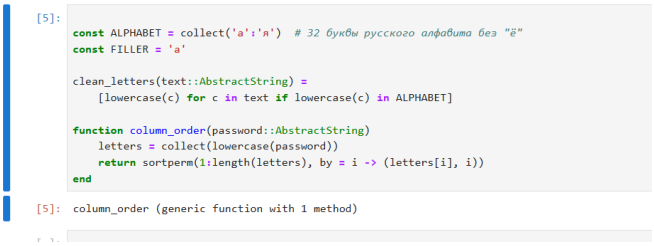


Рис. 5.1: Общая для всех трёх шифров функция

Во всех трёх реализациях используется русский алфавит из 32 букв *а–я* (без *ё*), как и в лабораторной работе №1. Служебная функция `clean_letters` оставляет от произвольной строки только буквы этого алфавита, переводя их в нижний регистр – перестановочные шифры оперируют фиксированным числом позиций, поэтому пробелы и знаки препинания из текста перед шифрованием удаляются. Порядок считывания столбцов по паролю вычисляется одной общей функцией `column_order`, переиспользуемой во всех трёх заданиях:

```
const ALPHABET = collect('а':'я')  # 32 буквы русского алфавита без "ё"
const FILLER = 'а'

clean_letters(text::AbstractString) =
    [lowercase(c) for c in text if lowercase(c) in ALPHABET]

function column_order(password::AbstractString)
    letters = collect(lowercase(password))
```

```
    return sortperm(1:length(letters), by = i -> (letters[i], i))
end
```

5.1 Маршрутное шифрование

```
function build_table(padded::Vector{Char}, m::Integer, n::Integer)
    table = Array{Char}(undef, m, n)
    for r in 1:m, c in 1:n
        table[r, c] = padded[(r - 1) * n + c]
    end
    return table
end

function route_encrypt(
    text::AbstractString,
    n::Integer,
    password::AbstractString,
)
    letters = clean_letters(text)
    m = cld(length(letters), n)
    padded = vcat(letters, fill(FILLER, m * n - length(letters)))
    table = build_table(padded, m, n)
    order = column_order(password)

    io = IOBuffer()
    for c in order, r in 1:m
        print(io, table[r, c])
    end
    return String(take!(io)), table
end

function route_decrypt(
    ciphertext::AbstractString,
    n::Integer,
    password::AbstractString,
)
    letters = clean_letters(ciphertext)
    m = div(length(letters), n)
    order = column_order(password)

    table = Array{Char}(undef, m, n)
    pos = 1
    for c in order, r in 1:m
        table[r, c] = letters[pos]
        pos += 1
    end

    io = IOBuffer()
    for r in 1:m, c in 1:n
        print(io, table[r, c])
```

Рис. 5.2: зеркальная операция

Текст дополняется буквой-заполнителем a до длины, кратной n, после чего построчно вписывается в таблицу $m \times n$ функцией `build_table`. Шифрование и расшифрование сводятся к чтению этой таблицы по столбцам либо, наоборот, к её заполнению по столбцам с последующим построчным чтением:

```
function build_table(padded::Vector{Char}, m::Integer, n::Integer)
    table = Array{Char}(undef, m, n)
```

```
    for r in 1:m, c in 1:n
        table[r, c] = padded[(r - 1) * n + c]
    end
    return table
end

function route_encrypt(
    text::AbstractString,
    n::Integer,
    password::AbstractString,
)
    letters = clean_letters(text)
    m = cld(length(letters), n)
    padded = vcat(letters, fill(FILLER, m * n - length(letters)))
    table = build_table(padded, m, n)
    order = column_order(password)

    io = IOBuffer()
    for c in order, r in 1:m
        print(io, table[r, c])
    end
    return String(take!(io)), table
end

function route_decrypt(
    ciphertext::AbstractString,
    n::Integer,
    password::AbstractString,
)
```

```
    letters = clean_letters(ciphertext)
    m = div(length(letters), n)
    order = column_order(password)

    table = Array{Char}(undef, m, n)
    pos = 1
    for c in order, r in 1:m
        table[r, c] = letters[pos]
        pos += 1
    end

    io = IOBuffer()
    for r in 1:m, c in 1:n
        print(io, table[r, c])
    end
    return String(take!(io))
end
```

`route_decrypt` – зеркальное отражение `route_encrypt`: сначала по паролю восстанавливается порядок столбцов и таблица заполняется по нему, а затем результат читается построчно.

5.2 Решётка Флейснера

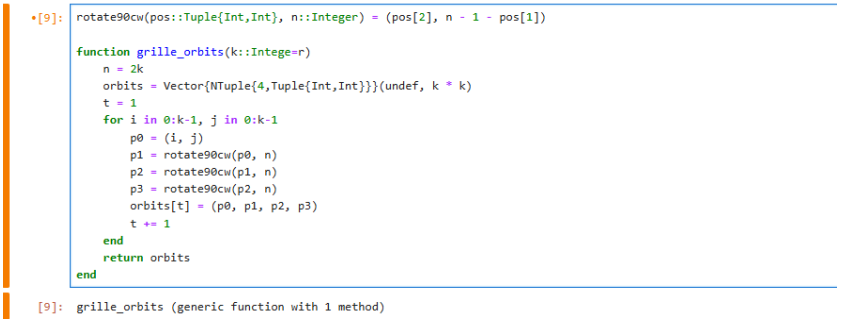


Рис. 5.3: Идея: поворот и орбиты

Поворот координаты клетки на 90° по часовой стрелке в квадрате размерности n (0-индексация) задаётся формулой $(i, j) \mapsto (j, n-1-i)$. Функция `grille_orbits` строит для решётки размерности k все k^2 орбит – по 4 клетки большого квадрата $2k \times 2k$, связанные поворотом:

```
rotate90cw(pos::Tuple{Int,Int}, n::Integer) = (pos[2], n - 1 - pos[1])


function grille_orbits(k::Integer)
    n = 2k
    orbits = Vector{NTuple{4,Tuple{Int,Int}}}(undef, k * k)
    t = 1
    for i in 0:k-1, j in 0:k-1
        p0 = (i, j)
        p1 = rotate90cw(p0, n)
        p2 = rotate90cw(p1, n)
        p3 = rotate90cw(p2, n)
        orbits[t] = (p0, p1, p2, p3)
        t += 1
    end
    return orbits
end
```

При шифровании буквы вписываются в клетки большого квадрата в порядке “4 положения поворота решётки $\times\, k^2$ прорезей”, что в точности моделирует физический процесс наложения решётки и её поворотов, после чего результат читается по столбцам, как и в маршрутном шифровании (пароль должен содержать ровно $2k$ неповторяющихся букв – по числу столбцов большого квадрата):

```
[11]: function grille_encrypt(
          text::AbstractString,
          k::Integer,
          password::AbstractString,
      )
          n = 2k
          letters = clean_letters(text)
          total = n * n
          padded = vcat(letters, fill(FILLER, total - length(letters)))

          orbits = grille_orbits(k)
          grid = Array{Char}(undef, n, n)
          idx = 1
          for s in 1:4, t in 1:k*k
              i, j = orbits[t][s]
              grid[i + 1, j + 1] = padded[idx]
              idx += 1
          end

          order = column_order(password)
          io = IOBuffer()
          for c in order, r in 1:n
              print(io, grid[r, c])
          end
          return String(take!(io)), grid
      end

[11]: grille_encrypt (generic function with 1 method)
```

Рис. 5.4: Заполнение “4 поворота x k² прорезей”

```
function grille_encrypt(
    text::AbstractString,
    k::Integer,
    password::AbstractString,
)
    n = 2k
    letters = clean_letters(text)
    total = n * n
    padded = vcat(letters, fill(FILLER, total - length(letters)))

    orbits = grille_orbits(k)
    grid = Array{Char}(undef, n, n)
```

```
    idx = 1
    for s in 1:4, t in 1:k*k
        i, j = orbits[t][s]
        grid[i + 1, j + 1] = padded[idx]
        idx += 1
    end

    order = column_order(password)
    io = IOBuffer()
    for c in order, r in 1:n
        print(io, grid[r, c])
    end
    return String(take!(io)), grid
end
```

grille_decrypt действует в обратном порядке: по паролю восстанавливает большой квадрат из криптограммы, а затем читает его клетки в той же последовательности орбит (`s, t`), в которой они заполнялись при шифровании, тем самым восстанавливая исходный порядок букв.

5.3 Шифр Виженера

```
function vigenere_shift(
    text::AbstractString,
    key::AbstractString,
    sign::Integer,
)
    pos = Dict(c => i - 1 for (i, c) in enumerate(ALPHABET))
    keyletters = [lowercase(c) for c in key if lowercase(c) in ALPHABET]
    m = length(ALPHABET)

    io = IOBuffer()
    ki = 0
    for ch in text
        l = lowercase(ch)
        if haskey(pos, l)
            shift = pos[keyletters[ki % length(keyletters) + 1]]
            j = mod(pos[l] + sign * shift, m)
            newch = ALPHABET[j + 1]
            print(io, isuppercase(ch) ? uppercase(newch) : newch)
            ki += 1
        else
            print(io, ch)
        end
    end
    return String(take!(io))
end

vigenere_encrypt(text::AbstractString, key::AbstractString) =
    vigenere_shift(text, key, 1)
vigenere_decrypt(text::AbstractString, key::AbstractString) =
    vigenere_shift(text, key, -1)
```

```
vigenere_decrypt (generic function with 1 method)
```

Рис. 5.5: Сдвиг с параметром знака

Сдвиг и обратный сдвиг реализованы одной функцией с параметром знака `sign` (аналогично тому, как в лабораторной работе №1 расшифрование Цезаря переиспользовало шифрование с обратным ключом): буквы пароля циклически сопоставляются буквам текста, а сдвиг каждой буквы текста равен номеру соответствующей буквы пароля в алфавите:

“‘{.julia} function vigenere_shift(text::AbstractString, key::AbstractString, sign::Integer,) pos = Dict(c => i - 1 for (i, c) in enumerate(ALPHABET)) keyletters = [lowercase(c) for c in key if lowercase(c) in ALPHABET] m = length(ALPHABET)

```
io = IOBuffer()
```

```
ki = 0
for ch in text
    l = lowercase(ch)
    if haskey(pos, l)
        shift = pos[keyletters[ki % length(keyletters) + 1]]
        j = mod(pos[l] + sign * shift, m)
        newch = ALPHABET[j + 1]
        print(io, isuppercase(ch) ? uppercase(newch) : newch)
        ki += 1
    else
        print(io, ch)
    end
end
return String(take!(io))
```

end

vigenere_encrypt(text::AbstractString, key::AbstractString) = vigenere_shift(text, key, 1) vigenere_decrypt(text::AbstractString, key::AbstractString) = vigenere_shift(text, key, -1) “‘.

6 Выводы

В результате выполнения лабораторной работы изучены и реализованы на языке Julia три шифра перестановки: маршрутное (столбцовое) шифрование, решётка Флейснера и шифр Виженера. Показано, что, несмотря на разный визуальный “механизм” (таблица, вращающаяся решётка, циклический сдвиг по алфавиту), маршрутное шифрование и решётка Флейснера сведены к общей операции – считыванию/заполнению прямоугольной таблицы по столбцам в порядке, заданном паролем; решётка Флейснера при этом лишь добавляет промежуточный шаг заполнения большого квадрата через орбиты поворота на 90°. Шифр Виженера, в свою очередь, устроен как параметризованное паролем обобщение шифра Цезаря из лабораторной работы №1: каждая буква сдвигается на свою величину, определяемую соответствующей буквой пароля. Работоспособность всех реализаций подтверждена экспериментально: во всех контрольных примерах шифрование с последующим расшифрованием восстанавливает исходный текст, а полученные криптограммы совпадают с примерами из методических указаний в тех фрагментах, для которых возможна прямая проверка.

Список литературы